BALAJI POINTS

Complete Project Architecture & UI/UX Blueprint

Multi-tenant • Multi-branch • Multi-role • Firebase • Flutter Material 3

Project purpose	Evolve the existing single-branch Carpenter Rewards app into a reusable multi-business, multi-branch platform.
Existing operation	Balaji Points – E Road branch, approximately 170 existing carpenter accounts using mobile number + 4-digit PIN.
Target capacity	Application-enforced target of up to 1,000 active and pending users per branch.
Authentication target	Email/password, Google Sign-In, mandatory mobile number and OTP verification, email verification.
Design system	Flutter Material 3, premium Navy + Gold palette, Light + Dark themes, responsive mobile/tablet/desktop/web.

Prepared: 1 November 2026 | Architecture proposal for planning and implementation

Document guide

Section	Contents
1	Executive summary and product goals
2	Architecture principles and tenant model
3	Firebase database architecture
4	Roles, memberships and permissions
5	Authentication, signup and account security
6	Branch isolation and security controls
7	Bills, points, rewards and approval workflows
8	Diwali closing, backup and branch reset
9	Existing 170-user E Road transition
10	Flutter Material 3 design system
11	Screen inventory and navigation flows
12	Performance, scalability and capacity
13	Suggested Flutter code architecture
14	Implementation phases and acceptance tests
15	Risks, decisions and launch checklist

Important scope note: This is an architecture and design blueprint, not a claim that the Firebase project, database, rules, Cloud Functions or Flutter code have already been changed. Validate Firebase quotas, pricing and SDK support against the actual project before implementation.

1. Executive summary and product goals

Balaji Points is planned as a reusable rewards and bill-management platform for hardware and plywood businesses. Each business (tenant) can operate one or more branches. Each branch has its own users, bills, points ledger, redemptions and branch-level administration. A central platform Super Admin manages tenants, branches, global roles, subscription/capacity settings and operational controls.

Core product goals

- Support a central Super Admin with visibility across all businesses and branches.
- Keep branch data isolated: a Branch Admin sees only their assigned branch; a carpenter sees only their own account, bills and points.
- Support dynamic roles and permissions rather than hardcoding every screen around two roles.
- Support email/password and Google Sign-In, with mandatory verified mobile number and verified email for new accounts.
- Preserve historical purchase, bill, points and prize records when a rewards cycle is closed.
- Provide safe, branch-wise selective reset tools with mandatory backup and audit logging.
- Keep Flutter UI responsive and smooth for a target of 1,000 users per branch.

Recommended platform model

Use a tenant layer now, even if Balaji Points initially has only one business. This prevents a costly redesign when the product is sold to additional shops. The tenant owns its branches; each branch has its own branch database according to the requested isolation model.

Key terminology

Term	Meaning
Platform Super Admin	Operator of the Balaji Points SaaS/platform; can manage all tenants and branches.
Tenant / Business	A customer business using Balaji Points, e.g. a hardware/plywood retailer.
Branch	A location under a tenant, e.g. E Road.
Branch Admin	Administrator limited to one or more explicitly assigned branches.
Carpenter	A branch member who submits bills and earns/redeems points.
Rewards cycle	A defined earning period, e.g. Diwali 2026, with its own closing and reporting.
Master database	Central registry for tenants, branches, identities, memberships and global configuration.
Branch database	Firestore database dedicated to one branch's operational records.

2. Architecture principles and tenant model

The recommended logical hierarchy is Platform → Tenant/Business → Branch → Membership/Role → Branch data. Branch identity and role assignment must be explicit. Do not infer access from a branch name, phone number, UI selection or a client-supplied database ID.

```
PLATFORM
■■■ TENANT / BUSINESS
■■■ Branch E Road
■ ■■■ Branch Admin / Manager / Accountant
■ ■■■ Carpenters
■ ■■■ Branch Firestore database
■■■ Branch Ahmedabad
■ ■■■ Branch Admin / Manager / Carpenters
■ ■■■ Branch Firestore database
■■■ Branch Rajkot
■■■ Branch Firestore database

CENTRAL MASTER DATABASE
■■■ tenants
■■■ branches
■■■ global_users
■■■ branch_memberships
■■■ role_definitions
■■■ permission_definitions
■■■ invitations / registration_requests
■■■ platform_audit_logs
```

Architecture rules

- Each branch has exactly one configured branch database ID in the trusted branch registry.
- Branch database selection is resolved by trusted backend data after authentication and membership validation.
- Client-side branch filters improve query efficiency but are not a security boundary.
- Platform Super Admin access is implemented through trusted backend services and explicit authorization.
- Branch Admin cannot grant themselves a role, change their branch, or access another branch database.
- Carpenter self-registration may request only the Carpenter role; elevated roles are assigned by authorized administrators.
- Where a user belongs to more than one branch, each branch assignment has a separate membership and role. A normal carpenter account should have one active branch unless the business explicitly allows otherwise.

Important product distinction

The requested 'one database per branch' model is separate from a tenant-per-business model. The master database should record tenantId, branchId and databaseId. Confirm current Firestore multi-database limits, quotas, billing, supported SDK behavior and operational tooling before committing to a large number of branches. If the expected number of branches may exceed the supported per-project database count, use separate Firebase projects or another partitioning strategy rather than silently exceeding platform limits.

3. Firebase database architecture

Proposed Firebase services

Service	Responsibility
Firebase Authentication	Email/password and Google identity. Phone Auth OTP can verify the mandatory mobile number.
Cloud Firestore – master_db	Tenant/branch registry, user profile, memberships, role definitions, permissions, invitations, capacity counters and platform audit metadata.
Cloud Firestore – branch database	Branch users/projections, bills, bill items, points ledger, rewards, redemptions, branch settings and branch audit events.
Cloud Storage	Bill images, attachments, export files and backup artifacts, with branch-scoped paths and access controls.
Cloud Functions / trusted backend	Registration orchestration, branch provisioning, bill approval, points calculation, reset jobs, exports and privileged operations.
App Check	Reduce unauthorized app/client access; complementary to Auth and security rules.
Crashlytics / Performance Monitoring	Crash reporting, app performance and production diagnostics.

Master database collections

```
master_db
■■■ tenants/{tenantId}
■ name, status, plan, createdAt
■■■ branches/{branchId}
■ tenantId, name, code, databaseId, status, userLimit
■■■ global_users/{uid}
■ name, email, phoneE164, emailVerified, phoneVerified, status
■■■ branch_memberships/{membershipId}
■ uid, tenantId, branchId, roleId, status, createdAt
■■■ role_definitions/{roleId}
■ tenantId or platform scope, name, permissionIds, status
■■■ permission_definitions/{permissionId}
■ module, action, description
■■■ registration_requests/{requestId}
■ uid, tenantId, branchId, requestedRoleId, status
■■■ branch_capacity/{branchId}
■ activeCount, pendingCount, limit, updatedAt
■■■ platform_audit_logs/{eventId}
actorUid, action, target, timestamp, metadata
```

Branch database collections

```
branch_<branch-key>
■■■ users/{uid}
■ branch-local profile/projection, status, role display fields
■■■ bills/{billId}
■ billNumber, carpenterUid, amount, status, cycleId, timestamps
■■■ bill_items/{itemId} (if item-level detail is required)
■■■ points_transactions/{transactionId}
■ uid, type, pointsDelta, billId, cycleId, createdAt
■■■ rewards/{rewardId}
■■■ reward_redemptions/{redemptionId}
■■■ cycles/{cycleId}
■ name, startAt, endAt, status, closingSnapshot
■■■ branch_statistics/{summaryId}
■■■ settings/{documentId}
■■■ audit_logs/{eventId}
```

Data ownership

The master database is the source of truth for global identity, tenant, branch and membership. The branch database is the source of truth for that branch's operational bills and rewards activity. Avoid maintaining independently editable copies of the same authoritative field in both places. If a branch-local user projection is needed for fast lists, synchronize it through trusted backend code.

Use stable IDs

Use Firebase Auth UID as the canonical user identity after the new authentication rollout. Keep legacyUserId fields and a migration mapping for old phone/document-ID-based accounts. Keep immutable bill IDs and transaction IDs. Never use a phone number as the permanent primary key for new records.

4. Roles, memberships and permissions

Recommended initial roles

Role	Scope	Typical access
Platform Super Admin	All tenants and branches	Create tenants/branches, provision databases, global roles, global reports, backup/reset orchestration.
Tenant Admin	One tenant	Manage tenant branches, tenant staff, tenant settings and tenant reports; no platform-wide access.
Branch Admin	Assigned branch	Manage branch users, bills, approvals, rewards, reports and branch settings.
Manager	Assigned branch	Operational management; configurable user and bill permissions.
Accountant	Assigned branch	Bills, payment/financial views, exports and transaction reports.
Supervisor	Assigned branch	Review bills, monitor carpenters and operational progress.
Carpenter	Assigned branch	Submit own bills, view own points, transactions and rewards.
Viewer	Assigned scope	Read-only access to explicitly allowed modules.

Permission model

Use permission identifiers such as users.view, users.create, bills.approve, points.adjust, rewards.manage, reports.export and branch.settings.manage. A role is a named bundle of permissions. UI visibility is driven by permissions, but every backend operation independently validates the permission and scope.

```
Example role document
{
"roleId": "branch_manager",
"name": "Manager",
"permissions": [
"dashboard.view",
"users.view",
"users.create",
"bills.view",
"bills.create",
"bills.approve",
"points.view",
"reports.view"
],
"scope": "branch",
"status": "active"
}
```

Branch membership example

```
{
"uid": "firebase_auth_uid",
"tenantId": "tenant_balaji",
"branchId": "branch_e_road",
"roleId": "carpenter",
"status": "active",
"joinedAt": "server_timestamp"
}
```

Critical rules

- Do not store a single global role field as the sole authorization source; role is assigned through membership and scope.

- Self-registration can select a branch and request Carpenter access only. The backend validates the branch is active and accepts registrations.
- Branch Admin can manage only memberships where branchId matches an assigned branch and where the requested action is permitted.
- Only Platform Super Admin can create branches and provision databases. Tenant Admin permissions, if enabled, must be tenant-scoped.
- Role changes, branch transfers, deactivation and permission changes must be audited.
- Do not let users write their own roleId, branchId, tenantId, points balance or approval status.

5. Authentication, signup and account security

Supported sign-in methods

- Email + Password.
- Google Sign-In.
- Mandatory mobile number for every signup.
- Mandatory mobile OTP verification to prove control of the mobile number.
- Mandatory email verification before full account activation.
- Password reset through Firebase email action link.
- Multi-factor authentication recommended/required for Platform Super Admin and Branch Admin, subject to the chosen Firebase Authentication / Identity Platform configuration.

New carpenter signup flow

1. User chooses Create Account.
2. User registers with email/password OR continues with Google.
3. Collect full name and mandatory mobile number in international E.164 format.
4. Send and verify OTP using Firebase Phone Authentication.
5. Verify email address using Firebase email verification.
6. User selects an active branch from a server-provided branch list.
7. Public signup role is fixed to Carpenter (or shown as Carpenter and not freely editable).
8. Backend checks branch capacity, duplicate verified phone, tenant/branch status and registration policy.
9. Create registration request and membership in pending status.
10. Branch Admin approves if approval is enabled; otherwise backend activates according to policy.
11. On activation, application loads only the assigned branch context.

Existing-user sign-in and Google linking

For existing users, link Google and password credentials to the same Firebase Auth UID where possible. If the old PIN system has no Firebase Auth identity, create a new Auth identity and preserve a permanent legacy-to-new UID mapping. Do not create a second business identity when the same verified phone/email already belongs to an account. Account linking and duplicate resolution must be performed through a secure, verified recovery flow.

Mobile number policy

- Normalize and store phone numbers in E.164 format, e.g. +919876543210.
- Maintain a unique verified-phone registry at tenant or platform scope according to the product's chosen uniqueness policy.
- Do not rely on an ordinary Firestore query alone to guarantee uniqueness during concurrent signups; use a trusted transaction/reservation mechanism.
- Changing a mobile number requires re-verification and an audit event.
- Do not use phone number as a password or expose OTPs in logs.

Password and session security

- Set a strong password policy; 12 characters is a recommended product baseline, not a Firebase universal requirement.

- Use Firebase Auth SDK flows; never store passwords or 4-digit PINs in Firestore.
- Apply abuse controls, OTP throttling, login monitoring and App Check.
- Revoke sessions/refresh tokens for compromised or deactivated accounts when appropriate.
- Clear branch-specific in-memory state and sensitive local caches on logout or account/branch switch.
- Do not treat biometric app lock as a replacement for Firebase authentication.

6. Branch isolation and security controls

Required behavior

Actor	Allowed	Must be denied
Carpenter A	Own profile, own bills, own points and eligible rewards in Branch A.	Other carpenters' private records, Branch B data, admin tools, points adjustments.
Branch Admin A	Users, bills, approvals, reports and settings in Branch A, within assigned permissions.	Any Branch B users, bills, points, transactions or settings.
Tenant Admin	Branches and data inside their tenant, only where explicitly granted.	Other tenants and platform-level controls.
Platform Super Admin	Cross-tenant administration through privileged backend workflows.	Unlogged or unaudited destructive operations; direct client bypass of security.

Security enforcement layers

1. Firebase Authentication verifies the identity.
2. Master membership lookup establishes tenant, branch and role assignments.
3. Trusted backend authorizes sensitive actions and resolves the branch database.
4. Firestore Security Rules protect client access within each database.
5. Storage Rules protect branch-specific bill images and attachments.
6. Audit logs record privileged actions and sensitive changes.

Important multiple-database caveat

Each Firestore database has its own rules configuration. Deploy and test the correct rules for every branch database. Firebase Admin SDK and server libraries bypass Firestore Security Rules, so every Cloud Function or server endpoint must implement its own authorization checks before using privileged access. Never assume that a branchId filter in Flutter is sufficient.

Client database selection

```
// Illustrative FlutterFire pattern.
// databaseId must come from a trusted, validated branch registry.
final branchDb = FirebaseFirestore.instanceFor(
app: Firebase.app(),
databaseId: trustedBranch.databaseId,
);
```

Rules and tests

- Default deny; grant only explicitly required operations.
- Validate authenticated UID, membership status, branch scope, role permission and record ownership.
- Test direct document reads, collection queries, writes, updates and deletes.
- Test malicious clients that replace branch IDs, database IDs, UIDs or role IDs.
- Test disabled users, removed memberships, expired invitations and switched accounts.
- Use Firebase Emulator Suite and automated integration tests before production.

7. Bills, points, rewards and approval workflows

Bill lifecycle

```
draft → submitted → under_review → approved
                                 → rejected
approved → points_posted
approved → (if policy allows) reversed with compensating entry
```

Bill data fields

Field	Purpose
billId	Immutable unique bill identifier.
tenantId / branchId	Ownership and isolation context.
carpenterUid	Canonical user who submitted the bill.
billNumber	Business-facing bill number; define uniqueness scope.
billDate / submittedAt	Purchase date and server submission time.
billAmount / tax / total	Validated monetary values.
imageStoragePath	Storage path; not a public unrestricted URL.
status	Draft, submitted, under_review, approved, rejected, reversed.
cycleId	Rewards cycle to which the bill belongs.
approvedBy / approvedAt	Auditable approval metadata.
pointsAwarded	Server-calculated points, not client-authoritative.

Points ledger design

Use an append-only points ledger. A current balance can be maintained as a cached summary, but every earning, redemption, adjustment and reversal must have an immutable transaction record. Never simply overwrite a balance without a corresponding ledger event.

```
points_transactions/{transactionId}
{
"uid": "carpenter_uid",
"branchId": "branch_e_road",
"cycleId": "diwali_2027",
"type": "bill_earning",
"pointsDelta": 150,
"billId": "bill_001",
"source": "approved_bill",
"createdAt": "server_timestamp",
"createdBy": "trusted_backend"
}
```

Approval and points workflow

1. Carpenter submits a bill and image.
2. Backend validates branch membership, bill fields, file ownership and duplicate checks.
3. Bill enters submitted/pending status.
4. Authorized Branch Admin/Manager reviews the bill.
5. On approval, trusted backend calculates points using the active branch reward rules and cycle.
6. Bill status and points ledger entry are committed idempotently so retries cannot award points twice.
7. Cached user balance and branch aggregate statistics are updated.
8. Carpenter receives notification and sees updated wallet/history.

Reward redemption

- Check available balance and reward eligibility on the backend.
- Reserve or deduct points atomically with creation of the redemption record.
- Record fulfilment status and acknowledgement.
- Use compensating ledger entries if a redemption is cancelled; do not silently delete ledger history.
- Prevent redemption during a branch reset or cycle-closing lock.

8. Diwali closing, backup and branch reset

The product requires a Super Admin feature to close a rewards cycle and selectively reset one branch. The user specifically wants points reset to zero, bill history cleared and related cycle data reset branch by branch. The safe default is to archive the old cycle and start a new cycle rather than physically destroy records.

Reset options

Operation	Effect	Default recommendation
Reset points	Set new-cycle available balance to zero; preserve old ledger in archived cycle.	Use cycle-based balance; avoid deleting historic ledger.
Clear bill history	Remove from active operational views or archive old bills and bill items.	Archive; do not permanently delete by default.
Reset points transactions	Close old cycle ledger and create a new active-cycle ledger view.	Retain historical ledger.
Clear redemptions	Close and reconcile pending/fulfilled redemptions; archive outcomes.	Never discard prize accountability.
Reset reward catalogue	Optional reset of branch reward catalogue/configuration.	Off by default; keep catalogue unless intentionally changing it.
Full branch cycle reset	Run selected reset steps for one branch only.	Require verified backup, confirmation and audit.

Recommended cycle model

```
branch_e_road
■■■ users/ KEEP
■■■ settings/ KEEP
■■■ role / membership registry KEEP (master database)
■■■ cycles/
■ ■■■ diwali_2026 archived / closed
■ ■■■ diwali_2027 active
■■■ bills/ tagged with cycleId
■■■ points_transactions/ tagged with cycleId
■■■ redemptions/ tagged with cycleId
■■■ rewards/ retained or intentionally revised
```

Safe reset job

1. Super Admin selects one branch and one or more reset actions.
2. Require recent re-authentication/MFA and explicit typed confirmation containing the branch code.
3. Lock bill submission, approval, point adjustments and redemptions for that branch.
4. Resolve all pending bills and redemptions; require a documented decision for each.
5. Create a full export/backup of master references, branch collections and Storage objects.
6. Verify backup manifests, record counts, file sizes/checksums and successful sample reads; record backup location and timestamp.
7. Execute a resumable, paginated backend job. Use idempotent job IDs and checkpoints; do not assume one Firestore batch can delete an entire branch.
8. Create the new active rewards cycle and zero starting balances.
9. Write audit records, reconcile counts and run post-reset validation.
10. Unlock branch only after successful validation; otherwise keep it locked and raise an operational alert.

Never reset/delete

- Branch and tenant records, branch database registry and configuration.
- Firebase Auth identities, user profiles, phone/email verification, memberships and role assignments.

- Admin accounts and permissions.
- Historical prize distribution and acknowledgement records.
- Backup manifests and reset audit logs.

Permanent deletion policy

If a legal or business requirement later demands permanent deletion, treat it as a separate, explicitly approved data-retention operation with retention checks and backup-expiry policy. A normal rewards reset should not delete user accounts or identity data.

9. Existing 170-user E Road transition

Current state: approximately 170 E Road users registered using mobile number and a 4-digit PIN. The business plans to distribute Diwali prizes based on approved purchases/points, take a backup and then ask users to register again for the new cycle and authentication model.

Recommended approach

Because the business has chosen a fresh registration after prize distribution, this is a new-cycle onboarding rather than a forced account migration. Still, archive the old user and transaction data so prize calculations, disputes and historical reporting remain possible.

Transition sequence

1. Freeze old-cycle earning at the announced closing date/time.
2. Resolve all pending bill approvals, rejected bills, reversals and redemptions.
3. Generate a final carpenter-wise closing report from approved bills and ledger entries.
4. Reconcile purchase amount, points earned, points redeemed and closing eligible balance.
5. Record prize category, prize issued, date, recipient acknowledgement and issuing admin.
6. Export and verify a complete E Road backup, including user records, bills, bill details, points transactions, redemptions, reward configuration and Storage attachments.
7. Mark old cycle as archived/read-only; do not reuse its active balances.
8. Create a new cycle and open new registrations.
9. Existing users register with new email/password or Google identity, mandatory mobile OTP and branch selection.
10. Match returning users to archived identity using verified mobile and an explicit re-registration/reconciliation process, without automatically restoring old points.
11. New cycle starts with zero points. Old history remains accessible to authorized admins as archived history.

Old PIN handling

- Do not migrate a four-digit PIN into a password field.
- Do not store plaintext PINs in the backup or new user profile.
- Disable the old PIN authentication path after the published transition period.
- If old PIN verification is used to establish account ownership before closing, apply attempt limits and use a secure backend verification process; never expose stored PIN material.
- New users must use the new Firebase Auth methods and mandatory mobile verification.

Prize calculation controls

Define the exact prize formula before the closing report is produced. Specify whether eligibility is based on approved bill amount, points earned, net points after redemptions, prize slabs or a combination. The final report should show the underlying values and formula version so prize outcomes can be explained and audited.

10. Flutter Material 3 design system

Visual direction

Premium business UI using Navy + Gold + neutral surfaces. Use Material 3 components and semantic color roles rather than scattering raw colors through widgets. Support light and dark themes from the beginning.

Token	Value / guidance
Primary Navy	#1D2B6B
Secondary Gold	#D97706
Success	#16A34A
Background	#F8F9FB
Surface	#FFFFFF
Text primary	#101828
Text secondary	#667085
Font	Manrope
Display / headline	32 / 24 dp (adapt to Material 3 type scale)
Title	20 / 16 dp
Body	16 / 14 dp
Label	12 dp
Radius	8 / 12 / 16 dp
Button height	48 dp target
Input height	Approximately 52 dp
Spacing	4 dp base grid; 16–24 dp screen padding

Material 3 component standards

- Use ThemeData(useMaterial3: true), ColorScheme, TextTheme and component themes.
- Buttons: FilledButton for primary action, OutlinedButton for secondary, TextButton for low-emphasis actions.
- Inputs: TextField/TextFormField with consistent labels, helper/error text and validation.
- Navigation: NavigationBar on compact mobile, NavigationRail on medium layouts, NavigationDrawer/sidebar on desktop.
- Data: paginated lists, DataTable/DataTable2-style responsive patterns where appropriate, clear sorting and filtering.
- Feedback: SnackBar for transient messages, Dialog/AlertDialog for confirmations, dedicated full-screen error/empty states where needed.
- Accessibility: readable contrast, scalable text, semantic labels, keyboard navigation and adequate touch targets.

Responsive breakpoints (starting proposal)

Layout	Suggested behavior
Compact mobile	Single-column content, NavigationBar or Drawer, stacked cards, full-screen forms.
Medium/tablet	NavigationRail, two-column dashboards, adaptive detail panes.

Layout	Suggested behavior
Expanded desktop/web	Persistent sidebar, top app bar, constrained content width, tables with filters and detail panels.

Shared UI states

- Loading: skeletons or compact progress indicators; avoid blank screens.
- Empty: explain why no data exists and show a permission-appropriate next action.
- Error: concise explanation, retry action and safe diagnostic logging.
- Success: clear confirmation and next step.
- Offline: show cached/read-only status where supported; queue only operations designed for safe retry.
- Permission denied: explain access limitation without leaking whether another branch's record exists.

11. Screen inventory and navigation flows

The following inventory is the baseline design scope. Some screens share components and should be implemented as reusable templates. Each screen needs normal, loading, empty, error, validation and permission-denied states as applicable.

Area	Screens / screen groups
Authentication (10)	Welcome; Login; Email signup; Google continuation; Mobile number; OTP verification; Branch selection; Email verification notice; Forgot/reset password; Registration pending/approval result.
Platform Super Admin (15)	Global dashboard; Tenant list/detail/create/edit; Branch list/detail/create/edit; Database provisioning status; Global user directory; Membership detail; Role list/editor; Permission matrix; Global bills/report view; Platform analytics; Audit log; Capacity management; Backup center; Reset job monitor.
Branch management (8)	Branch overview; Branch profile; Branch admin assignment; Branch settings; Branch reward rules; Branch capacity; Branch status/activation; Branch activity/audit.
User and role management (8)	User list; User detail; Invite/create user; Edit profile; Membership/branch assignment; Role assignment; Role editor; Deactivate/reactivate user.
Branch Admin (12)	Branch dashboard; Carpenter list; Carpenter detail; Pending registrations; Bill list; Bill review; Approve/reject confirmation; Points adjustment request; Rewards management; Redemption list/detail; Branch reports; Branch settings.
Carpenter (10)	Home; Wallet; Add bill; Bill preview/submit; My bills; Bill detail/status; Points history; Rewards catalogue; Reward detail/redeem; Profile/settings.
Bills and approvals (7)	Bill list; Bill detail; Bill image viewer; Review/approve; Reject with reason; Duplicate/exception review; Export.
Rewards and points (7)	Reward catalogue; Reward editor; Points rules; Points ledger; Redemption list; Redemption detail/fulfilment; Cycle closing summary.
Reports (5)	Branch sales; Carpenter purchase ranking; Points liability/ledger; Reward redemption/prize distribution; Export center.
Settings/notifications (5)	Notification center; Notification preferences; Theme/language; Security/session settings; Help/support.

Navigation rules

- Role and permission determine navigation items; hidden menu items are not security enforcement.
- Super Admin has a tenant/branch context selector and a clearly visible global scope indicator.
- Branch Admin sees their branch name/code in the app bar; no cross-branch selector unless explicitly assigned multiple branches.
- Carpenter sees their own branch and their own wallet/bills only.
- On compact screens, use bottom navigation for Carpenter's primary five destinations; secondary actions live under Profile/More.
- On desktop, use a persistent sidebar and preserve page state when navigating between list and detail views.

Primary flow: add bill

```
Carpenter Home
→ Add Bill
→ Enter bill date / number / amount
→ Add bill image / items if required
→ Review
→ Submit
```

```
→ Submission confirmation
→ My Bills / Bill Detail
→ Pending → Approved or Rejected
→ Points ledger and wallet updated after approval
```

Primary flow: branch creation

```
Super Admin
→ Branch Management
→ Create Branch
→ Select tenant
→ Enter branch name/code/address
→ Provision branch database (backend)
→ Initialize schema/settings/rules
→ Assign Branch Admin
→ Activate branch
→ Verify health and access tests
```

12. Performance, scalability and capacity

Business capacity target

Enforce an initial maximum of 1,000 active + pending user accounts per branch as a business rule. This is not a Firestore platform maximum. Do not imply that Firestore is limited to 1,000 users. Keep the limit configurable per tenant/plan and enforce it server-side.

Area	Starting recommendation
User list	Fetch 20 records per page; cursor-based pagination.
Bills	Fetch 20 records per page; filter by status/date/carpenter; avoid unbounded listeners.
Points transactions	Fetch 30 records per page; order by server timestamp and stable document ID.
Rewards	Fetch only active rewards; paginate if catalogue grows.
Dashboard	Read precomputed aggregate documents; do not download all bills to calculate totals.
Images	Compress at upload, store thumbnails, lazy-load and cache.
Search	Debounce input; use indexed prefix/filter queries or a dedicated search service for full-text/fuzzy search.
Listeners	Use real-time listeners only where live updates materially help; scope and dispose them correctly.
Rendering	ListView.builder/SliverList, const widgets, narrow Riverpod providers, avoid broad rebuilds.
Offline	Define offline behavior explicitly; clear sensitive branch-specific app state on logout/switch.

Flutter rendering practices

- Use Riverpod providers scoped by feature and query parameters; avoid one global provider that rebuilds an entire screen.
- Use ListView.builder, SliverList and lazy image widgets for long lists.
- Keep expensive formatting/calculations out of build methods.
- Use const constructors where possible and split widgets into small independently updating components.
- Debounce search and cancel stale requests.
- Dispose controllers, subscriptions and streams when screens are removed.
- Measure in profile/release mode on representative low-end Android phones, not only debug mode.

Dashboard aggregation

```
branch_statistics/current
{
"activeUsers": 170,
"pendingUsers": 4,
"approvedBills": 2450,
"salesAmount": 12345678,
"pointsIssued": 250000,
"pointsRedeemed": 42000,
"updatedAt": "server_timestamp"
}
```

Capacity counter

Maintain a trusted branch capacity record. Registration should reserve capacity atomically, count active and pending registrations, and release capacity when a request is rejected or a user is deactivated according to the business policy. Prevent concurrent signup requests from exceeding the configured limit. Add monitoring when a branch approaches 80%, 90% and 100% capacity.

Performance targets to validate

Initial engineering targets: cached dashboard visible within about 2 seconds; first network-backed list page within about 3 seconds on a representative connection; maintain smooth frame rendering (target 60 FPS on supported devices). These are test targets, not guarantees. Define test devices, network profiles and percentile measurements before launch.

13. Suggested Flutter code architecture

Use feature-first architecture with a clear separation between presentation, application/state, domain and data layers. The current project has used Flutter, Riverpod and GoRouter; this proposal continues those patterns.

```
lib/
■■■ app/
■ ■■■ app.dart
■ ■■■ router.dart
■ ■■■ bootstrap.dart
■■■ core/
■ ■■■ auth/
■ ■■■ firebase/
■ ■ ■■■ firebase_initializer.dart
■ ■ ■■■ database_manager.dart
■ ■ ■■■ branch_context.dart
■ ■■■ permissions/
■ ■■■ theme/
■ ■ ■■■ app_theme.dart
■ ■ ■■■ app_colors.dart
■ ■ ■■■ app_typography.dart
■ ■ ■■■ app_spacing.dart
■ ■ ■■■ app_components.dart
■ ■■■ errors/
■ ■■■ pagination/
■ ■■■ logging/
■■■ features/
■ ■■■ authentication/
■ ■■■ super_admin/
■ ■■■ tenants/
■ ■■■ branches/
■ ■■■ memberships/
■ ■■■ roles_permissions/
■ ■■■ branch_admin/
■ ■■■ carpenters/
■ ■■■ bills/
■ ■■■ points/
■ ■■■ rewards/
■ ■■■ redemptions/
■ ■■■ reports/
■ ■■■ backups/
■ ■■■ cycle_management/
■■■ data/
■ ■■■ models/
■ ■■■ repositories/
■ ■■■ services/
■■■ shared/
■■■ widgets/
■■■ forms/
■■■ dialogs/
■■■ utils/
```

Core services

Service	Responsibility
AuthService	Email/password, Google sign-in, phone OTP verification, email verification, sign-out and linking.
MasterRepository	Tenant, branch, user identity and membership reads.
BranchContextService	Resolve current authorized branch and database ID; clear context on logout/switch.
DatabaseManager	Return master or validated branch Firestore instance; no arbitrary client-selected database IDs.
PermissionService	Resolve effective permissions for current membership; UI convenience only.
BillRepository	Paginated bill reads; submit request; detail and status.
PointsRepository	Read ledger and balance summary; no client-authoritative adjustments.

Service	Responsibility
ResetJobService	Request, monitor and report backend reset jobs.
AuditRepository	Read permitted audit views; writes for privileged events remain backend-controlled.

State management

Use Riverpod providers for authenticated user, active membership, branch context, permissions, paginated list state and feature-level data. Keep providers narrowly scoped. Use GoRouter redirect/guards for navigation, while backend authorization remains authoritative.

Illustrative theme setup

```
ThemeData buildLightTheme() {
final scheme = ColorScheme.fromSeed(
seedColor: const Color(0xFF1D2B6B),
brightness: Brightness.light,
).copyWith(
primary: const Color(0xFF1D2B6B),
secondary: const Color(0xFFD97706),
surface: const Color(0xFFFFFFFF),
);

return ThemeData(
useMaterial3: true,
colorScheme: scheme,
scaffoldBackgroundColor: const Color(0xFFF8F9FB),
fontFamily: 'Manrope',
);
}
```

14. Implementation phases and acceptance tests

Phase	Deliverables	Exit criteria
0. Discovery	Inspect current app, Firestore collections, Auth method, Storage, rules, indexes, Cloud Functions, backup and deployment setup.	Document actual schema and data dependencies; confirm Firebase project constraints.
1. UI foundation	Material 3 light/dark themes, typography, tokens, responsive shell, shared widgets and design states.	Approved screen layouts and component catalogue; responsive smoke tests pass.
2. Identity	Email/password, Google linking, mandatory phone OTP, email verification, signup requests and recovery.	Verified identity, duplicate handling and secure linking tests pass.
3. Tenant/branch core	Master DB schema, branch registry, branch provisioning, memberships and capacity counters.	Create branch end-to-end; users assigned only to valid branches.
4. Authorization	Roles, permission matrix, Firestore/Storage rules, backend authorization and audit logs.	Cross-branch negative tests pass for all roles and operations.
5. Business features	Bills, approvals, points ledger, rewards, redemptions, reports and notifications.	No duplicate points on retries; all operations are branch-scoped.
6. Diwali close/reset	Cycle close, closing reports, backup export/verification, reset job and archive.	Restore test passes; branch reset cannot affect another branch.
7. Existing data transition	E Road 170-user backup, prize reconciliation, archived cycle and fresh registration.	Counts and financial/points reconciliation signed off.
8. Performance/launch	Pagination, indexes, monitoring, load tests, device testing and staged rollout.	Performance and security acceptance checklist passed.

Mandatory acceptance tests

- Carpenter A cannot list, read, update or query Branch B users, bills, points or files.
- Branch Admin A cannot access Branch B records even by changing document IDs, query parameters or database selection.
- Carpenter can read only their own bills and points ledger.
- Self-registration cannot create Admin/Super Admin membership.
- Unverified phone or email cannot gain active access according to policy.
- Duplicate verified phone signup is blocked safely under concurrent requests.
- Branch capacity never exceeds the configured limit under concurrent signup load.
- Bill approval awards points exactly once, including retries and function redelivery.
- Rejected/reversed bills do not retain incorrectly awarded points; corrections use ledger entries.
- Branch reset of E Road leaves every other branch unchanged.
- Reset cannot start without a verified backup and explicit authorization.
- Failed reset remains locked/recoverable and exposes a clear job state.
- Historical Diwali prize reports and archived bills remain readable to authorized roles.
- Logout/account switch clears branch-specific sensitive state and caches.
- Long lists paginate and do not render all records at once.
- Light/dark themes, accessibility, keyboard use and compact/medium/expanded layouts are tested.

15. Risks, decisions and launch checklist

Key risks to resolve early

Risk	Mitigation
Firestore database-per-branch operational ceiling	Confirm current project limits and supported tooling. Define a project-per-tenant or sharding fallback if branch count can exceed limits.
Rules differ per Firestore database	Automate rules deployment and security tests for every provisioned branch database.
Admin SDK bypasses rules	Implement explicit backend authorization in every privileged function.
Old data uses phone IDs and mixed UIDs	Inventory references and build a legacy ID mapping before any migration or archive.
Duplicate accounts after fresh signup	Use verified phone matching and a reviewed re-registration reconciliation workflow.
Points balance drift	Append-only ledger, idempotency keys and scheduled reconciliation.
Destructive reset causes data loss	Mandatory backup, restore test, staged job, lock, checkpoints and audit trail.
Dashboard read costs grow	Precomputed aggregates, pagination and scoped listeners.
Multiple Auth providers create duplicate identities	Secure provider linking and verified account recovery.

Decisions to confirm before coding

- Will each customer business be a tenant with its own branches? Recommended: yes.
- Will each branch have a separate Firestore database inside one Firebase project, or will larger customers require separate Firebase projects? Validate limits before production.
- Can a staff member have memberships in multiple branches? Recommended: yes for staff, but one branch for ordinary carpenter self-registration.
- Is branch Admin approval required for every new carpenter? Define per-branch setting.
- Is mobile number unique platform-wide or unique within each tenant? Recommended: unique within a tenant unless the business needs cross-tenant identity.
- Which exact formula determines Diwali prizes: approved purchase value, points earned, net balance, slabs or a combination?
- Will old bill history be archived and retained, or permanently deleted under a separate retention policy? Recommended: archive.
- Which data may Branch Admin export, and which reports are Super Admin-only?

Production launch checklist

- Fresh backup and restore test completed.
- Firestore schema, indexes and rules reviewed.
- Cloud Functions authorization and idempotency reviewed.
- Storage rules tested against cross-branch access.
- Authentication provider linking and OTP abuse controls tested.
- Super Admin MFA enabled.
- Branch capacity and branch provisioning tested.
- All roles tested using separate test accounts.

- Reset tested in a staging branch with restore verification.
- Crashlytics and Performance Monitoring enabled.
- Privacy policy, terms, retention and user support process published.
- Staged rollout and rollback plan approved.

Final recommendation

Build in this order: (1) inspect and document the existing E Road Firebase implementation, (2) approve the Material 3 design system and screen flows, (3) implement tenant/branch identity and authorization, (4) implement authentication and registration, (5) migrate/close the old Diwali cycle, and (6) release business features with performance and security tests. Do not begin with a destructive database reset or mass account recreation.

Reference note

This document is a proposed implementation blueprint based on the requirements discussed. Firebase product limits, billing, multi-database behavior, FlutterFire APIs, Firebase Authentication provider linking and Identity Platform MFA availability can change. Verify the current official Firebase documentation and the actual Balaji Points project configuration during technical discovery.

End of architecture document